

K. Srinath

192504280

CSA1417 - Artificial Intelligence

Autonomous Vacuum-Cleaning Robot:

PEAS Description

PEAS stands for performance measure, Environment, Actuators and Sensors.

Component	Description
Performance measure	Keep both rooms clean, minimize movements, Save time and energy
Environment	Two rooms, each of which can be dirty
Actuators	move left, move right, Suck or Nothing
Sensors	Detect robot's current location

Explanation:

1. Performance measure: The robot should clean rooms efficiently
2. Environment: Two rooms, each of which can be dirty
3. Actuators: These allow robot to move between rooms and clean
4. Sensors: These provide information about robot's location and robot condition.

B. Type of intelligent Agent

A model Based reflex agent is suitable because it can maintain information about condition of both rooms

Reasoning:

1. It keeps track of robot's current location
2. It remembers the known condition of both rooms
3. If current room is dirty, it performs Suck Action
4. If current room is clean, it moves to other room
5. If both rooms are clean, it performs do Nothing.

Therefore, a model-based reflex agent can make better decisions by using current sensor information and internal state.

2. Properties of Task Environments

Property	Chess	Self-driving Car	Crossword puzzle
Observability	Fully observable	Partially observable	Fully observable
Determinism	Deterministic rules	Stochastic	Deterministic
Episodic/ Sequential	Sequential	Sequential	Sequential
Static/dynamic	Static	dynamic	Static
discrete/ continuous	Discrete	Continuous	Discrete

Explanation:

1. Chess: The entire board is visible, and legal moves have predictable outcomes. Each move affects future moves, so it is sequential and discrete.
2. SELF-driving cars: The car cannot observe every road condition or predict other drivers perfectly. Traffic changes continuously, making environment partially observable, dynamic and stochastic.
3. crossword puzzle: The grid and clues are visible. Each word affects other answers, so it is sequential.

3. Formation of delivery Robot System.

(i) Initial State:

Initial State is starting position of robot in building grid.

Ex: $S = (1, 1)$ meaning robot starts row 1, column 1.

(ii) Goal State / Goal Test:

The Goal State is delivery location.

Ex: $G = (5, 5)$

The goal test checks whether robot's current location matches delivery location.

(iii) Actions:

The robot can perform four actions:

- * up
- * down
- * left
- * Right

(iv) Transition model:

The transition model defines the robot's next position after an action.

v) Path cost function

Each valid movement has cost of 1

$$\text{Path cost} = \sum \text{cost of each move}$$

For example, path containing 6 moves has total cost of 6.

Justification: This model identifies where robot should start, where it must go, where it can perform, how action change its position, and how total cost of route is calculated.

4. Breadth-First Search (BFS):

BFS Approach:

BFS explores nodes level by level using FIFO queue.

Assume adjacent nodes are explored in Alphabetic order.

The graph has these edges:

S-A, S-B, A-C, A-D, B-E, C-F, E-G

Queue at Each Step:

Step	Expanded Node	Queue (Frontier)
1	S	[A, B]
2	A	[B, C, D]
3	B	[C, D, E]
4	C	[D, E, F]
5	D	[E, F]
6	E	[F, G]
7	F	[G]
8	G	[]

The search searches G from E

Final BFS path: $S \rightarrow B \rightarrow E \rightarrow G$

NO. OF edges = 3

path cost: $1+1+1=3$

BFS properties:

Property

Explanation

Time Complexity

$O(V+E)$ using an adjacency-list graph representation

Optimality

Optimal when all edges have an equal cost

Space Complexity

Complete for finite graphs when states avoided

Completeness

Complete for finite graphs

Here, v is no. of vertices and E is no. of Edges.

5. Depth-First Search (DFS):

DFS Approach:

DFS explores one branch as deeply as possible before returning to explore another branch. It uses LIFO Stack

Stack at Each Step

Step	Expanded Node	Stack
1	S	[B, A]
2	A	[B, D, C]
3	B	[B, D, F]
4	C	[B, D]
5	D	[B]
6	E	[E]
7	F	[G]
8	G	[]

Final DFS path: $S \rightarrow B \rightarrow E \rightarrow G$

No. of edges = 3

path cost = $1 + 1 = 2$

BFS and DFS Comparison:

Criterion	BFS	DFS
Data Structure	Queue (FIFO)	Stack (LIFO)
Search method	Level by Level	deep along a branch
Path Found	$S \rightarrow B \rightarrow E \rightarrow G$	$S \rightarrow B \rightarrow E \rightarrow G$
Path cost	3	3
Completeness	Complete for finite graphs with repeated state-check	Complete for finite graph with repeated statecheck
Optimality	Optimal for equal edge cost	Not generally optimal
memory usage	can be high for wide graphs	often lower